

Notes on Distributed Cache Invalidation

Overview

This note records how a small distributed cache keeps its entries consistent when several writers touch the same keys. It assumes no shared clock and no central coordinator, which is the setting most services actually run in.

The goal is not a proof but a shared vocabulary, so the later sections can refer back to the same terms without redefining them each time.

Cache Coherence

A read should never return a value older than one the same client has already observed. That single rule, monotonic reads, is weaker than linearizability but strong enough for session-scoped work, and it is cheap to provide with per-client version stamps.

Coherence across clients is a separate question and is handled by the invalidation path rather than by the read path.

Invalidation Strategies

Two families exist. Write-through invalidation evicts an entry the moment its backing record changes, trading write latency for freshness. Lease-based invalidation lets an entry live until its lease expires, trading freshness for far less coordination traffic.

The cache described here uses leases for hot keys and write-through for the small set of keys marked authoritative, because their staleness budget is effectively zero.

Write Propagation

When a record changes, the writer publishes the key and a monotonically increasing version to a fan-out topic. Each cache node applies the eviction only if the published version exceeds the version it currently holds, so reordered messages are harmless.

Propagation is best-effort. A node that misses a message will still converge once the entry lease expires, which bounds the worst-case staleness to the lease duration.

Failure Modes

The interesting failures are partial. A node partitioned from the fan-out topic keeps serving reads from leases that have not yet expired, so it can serve values other nodes have already invalidated. This is acceptable precisely because leases are short.

A writer that crashes after mutating the record but before publishing leaves every cache stale until the lease expires. Shortening the lease narrows this window at the cost of more misses.

Evaluation

On a synthetic workload with a ninety-ten read-write split, lease durations between two and ten seconds kept the observed staleness under one lease duration in every run, while coordination traffic fell by roughly an order of magnitude against write-through everywhere.

The authoritative-key set stayed small enough that its write-through cost was not measurable against the background load.

Closing Notes

None of this is novel; it is the folklore written down so the next person does not have to reconstruct it from the code. The one non-obvious point is that reordered invalidations are safe as long as versions are compared, never assumed.